

DESENVOLVIMENTO ÁGIL DE SOFTWARE



Diego Augusto Barros
ENSINAR É TECNOLOGIA

CONTEÚDO

1. Desenvolvimento Ágil de Software
2. Métodos Ágeis
3. Escalabilidade dos Métodos Ágeis

Diego Augusto Barros
ENSINAR É TECNOLOGIA



1

DESENVOLVIMENTO ÁGIL DE SOFTWARE

INTRODUÇÃO

Atualmente as empresas
operam em um **ambiente**
global em **rápida mudança**.



INTRODUÇÃO

As empresas precisam responder:

- Às novas oportunidades e mercados;
- Às mudanças nas condições econômicas;
- Ao surgimento de produtos e serviços concorrentes.



INTRODUÇÃO

Os sistemas fazem parte de
quase todas as operações
de negócios



INTRODUÇÃO

Novos sistemas devem ser **desenvolvidos rapidamente** para:

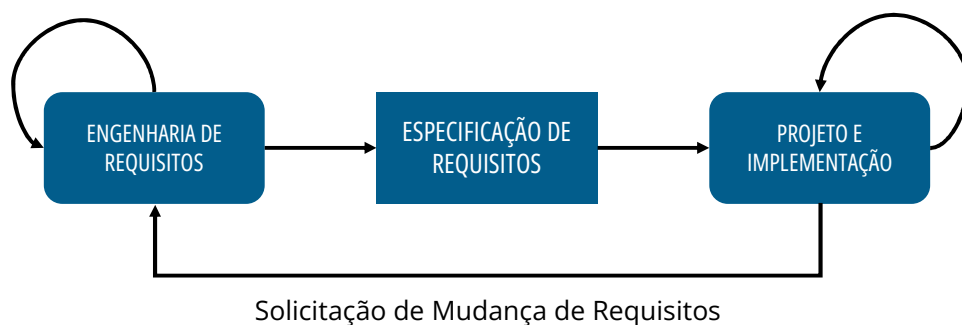
- Obter vantagem competitiva;
- Responder à pressão da concorrência.



A **entrega e desenvolvimento rápidos** é **crucial** para a maioria dos sistemas de negócios.

As empresas podem estar dispostas a sacrificar a qualidade e comprometer requisitos para uma implantação rápida do novo software.

PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS



PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS

Os processos de desenvolvimento de software dirigidos por plano que:

- Especificam completamente os requisitos;
- Depois projetam;
- Constroem e testam um sistema



Não são voltados para o desenvolvimento rápido de software.

PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS

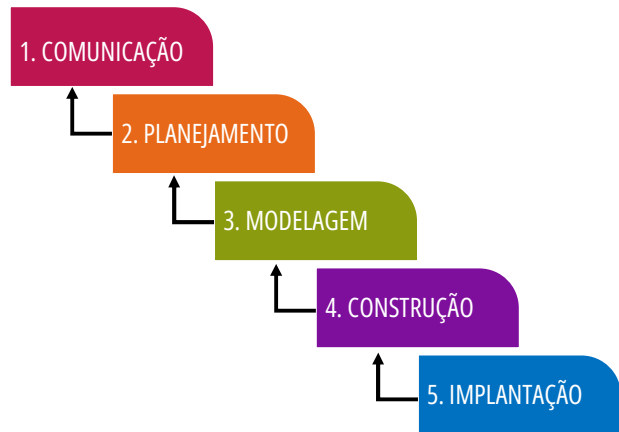
Mudanças ou problemas nos requisitos exigem retrabalho e testes adicionais.



Não são voltados para o desenvolvimento rápido de software.

EXEMPLO

- Processos de Software Prescritivos, por exemplo:
 - Método em Cascata;
 - Baseado em Especificações.
- São demorados, resultando em entregas tardias ao cliente.



PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS

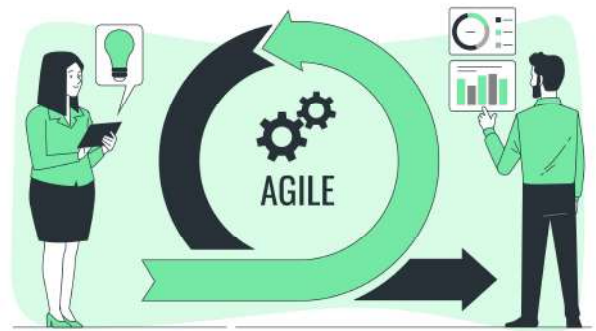
Indicado para sistemas onde uma análise completa é essencial, por exemplo:

- Sistemas de controle críticos em segurança



DESENVOLVIMENTO RÁPIDO DE SOFTWARE

- O desenvolvimento rápido de software é conhecido como:
 - Desenvolvimento Ágil
 - Métodos Ágeis.
- Esses métodos são concebidos para produzir software útil de maneira rápida.



CARACTERÍSTICAS DO DESENVOLVIMENTO ÁGIL

- Intercalação dos processos de especificação, projeto e implementação.
- Ausência de uma especificação detalhada do sistema.



CARACTERÍSTICAS DO DESENVOLVIMENTO ÁGIL

- Minimização ou geração automática da documentação do projeto pelo ambiente de programação.
- Documento de requisitos do usuário contém uma definição resumida com as características mais importantes do sistema



CARACTERÍSTICAS DO DESENVOLVIMENTO ÁGIL

- Desenvolvimento do sistema em incrementos.
- Participação de usuários finais e *stakeholders* na especificação e avaliação de cada incremento.
- Possibilidade de propostas de novos requisitos para implementação em versões futuras do sistema.



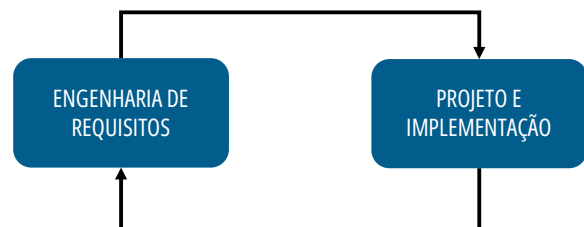
CARACTERÍSTICAS DO DESENVOLVIMENTO ÁGIL

- Ampla utilização de ferramentas par auxiliar no processo de desenvolvimento, por exemplo:
 - Testes Automatizados;
 - Gerenciamento de Configuração;
 - Integração de Sistemas;
 - Automação da interface do usuário.



PROCESSO DE SOFTWARE ÁGIL

- Projeto e implementação são atividades centrais do desenvolvimento
- Outras tarefas são incorporadas a essas atividades, por exemplo:
 - Elicitação de requisitos
 - Testes



EXEMPLO: PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS

- A iteração ocorre dentro das atividades;
- Com documentos formais, sendo usados como comunicação entre as etapas do processo;



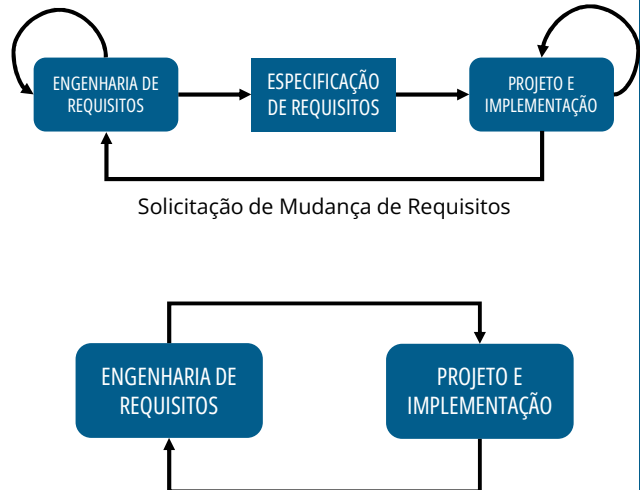
EXEMPLO: PROCESSO DE SOFTWARE DIRIGIDO POR PLANOS

- Os requisitos evoluirão;
- Resultando na produção de uma **especificação de requisitos**;
- Que servirá como **entrada** para o processo de **projeto e implementação**.



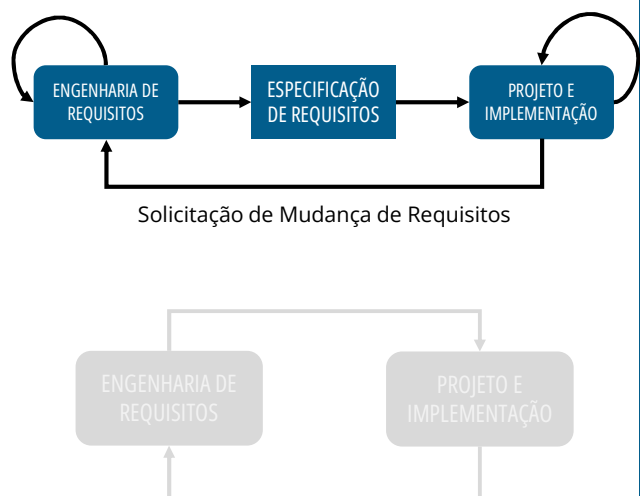
DIRIGIDO POR PLANOS X ÁGIL

- Os processos **dirigidos por plano** são utilizados frequentemente com **práticas de programação ágil**
- Os **métodos ágeis** podem incorporar algumas **atividades planejadas**, além da programação e dos testes



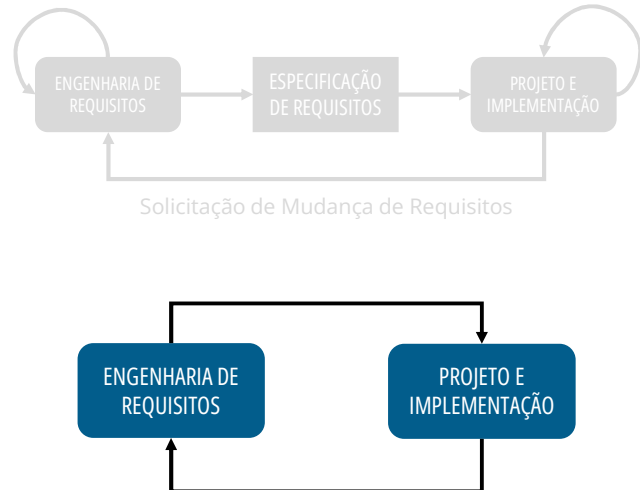
DIRIGIDO POR PLANOS X ÁGIL

É perfeitamente viável alocar requisitos e planejar a fase de projeto (design) e desenvolvimento como uma série de incrementos.



DIRIGIDO POR PLANOS X ÁGIL

Não é inevitavelmente focado no código e pode produzir alguma documentação de projeto (design).



DIRIGIDO POR PLANOS X ÁGIL

Os desenvolvedores podem decidir que uma iteração não produzirá código novo, mas sim modelos e documentação de sistema.



HISTÓRIA DO DESENVOLVIMENTO ÁGIL

Nos anos de 1980 e início dos anos 1990, havia uma visão generalizada de que a melhor maneira para conseguir o melhor software era por meio:

- Planejamento **cuidadoso** do projeto;
- Garantia de **qualidade formalizada**;
- Uso de métodos de análise e projeto apoiado por ferramentas de software CASE (*Computer-Aided Software Engineering*);
- Processo de desenvolvimento de software **rigorosos** e **controlados**.

HISTÓRIA DO DESENVOLVIMENTO ÁGIL

Essa percepção veio da comunidade de engenharia de software que era responsável por desenvolver sistemas grandes e duradouros, como os destinados aos setores aeroespacial e governamental.



HISTÓRIA DO DESENVOLVIMENTO ÁGIL

- Quando essa abordagem pesada de desenvolvimento é aplicada para sistemas de negócio de pequeno ou médio porte;
- A sobrecarga de trabalho é tão grande que domina o processo de desenvolvimento de software.



HISTÓRIA DO DESENVOLVIMENTO ÁGIL

- Mais tempo é dedicado à decisão de como o sistema deve ser desenvolvido do que na programação ou nos testes.
- Mudanças nos requisitos exigem retrabalho, levando a alterações na especificação e no design inicial do projeto.



HISTÓRIA DO DESENVOLVIMENTO ÁGIL

- A insatisfação com essas abordagens levou ao surgimento dos métodos ágeis no final da década de 1990.
- Eles permitiram que o time de desenvolvimento se concentrasse no próprio software, em vez de no projeto (design) ou na documentação.



ALIANÇA ÁGIL

Em 2001, Kent Beck e 16 outros notáveis desenvolvedores de software, escritores e consultores (referidos como "Agile Alliance") assinaram o "Manifesto para o Desenvolvimento Ágil de Software"



MANIFESTO ÁGIL



Estamos descobrindo maneiras melhores de desenvolver software,
fazendo-o nós mesmos e ajudando outros a fazerem o mesmo.

Através deste trabalho, passamos a valorizar:

Indivíduos e interações mais que *processos e ferramentas*
Software em funcionamento mais que *documentação abrangente*
Colaboração com o cliente mais que *negociação de contratos*
Responder a mudanças mais que *seguir um plano*

Ou seja, mesmo havendo valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

Estamos descobrindo maneiras melhores de desenvolver software,
fazendo-o nós mesmos e ajudando outros a fazerem o mesmo.

Através deste trabalho, passamos a valorizar:

Indivíduos e interações mais que *processos e ferramentas*
Software em funcionamento mais que *documentação abrangente*
Colaboração com o cliente mais que *negociação de contratos*
Responder a mudanças mais que *seguir um plano*

Ou seja, mesmo havendo valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

- | | | | |
|---|--|----|--|
| 1 | Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado. | 7 | Software funcionando é a medida primária de progresso. |
| 2 | Mudanças nos requisitos são bem-vindas, mesmo tardiamente no desenvolvimento. Processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente. | 8 | Os processos ágeis promovem desenvolvimento sustentável. Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente. |
| 3 | Entregar frequentemente software funcionando, de poucas semanas a poucos meses, com preferência à menor escala de tempo. | 9 | Continua atenção à excelência técnica e bom design aumenta a agilidade. |
| 4 | Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto. | 10 | Simplicidade: a arte de maximizar a quantidade de trabalho não realizado; é essencial. |
| 5 | Construa projetos em torno de indivíduos motivados. Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho. | 11 | As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis. |
| 6 | O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face. | 12 | Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e então refina e ajusta seu comportamento de acordo. |

OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

1. Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

2. Mudanças nos requisitos são bem-vindas, mesmo tardiamente no desenvolvimento.

Processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

3. Entregar frequentemente software funcionando, de poucas semanas a poucos meses, com preferência à menor escala de tempo.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

4. Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

5. Construa projetos em torno de indivíduos motivados.

Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

6. O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

7. Software funcionando é a medida primária de progresso.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

8. Os processos ágeis promovem desenvolvimento sustentável.

Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

9. Contínua atenção à excelência técnica e bom design aumenta a agilidade.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

10. Simplicidade: a arte de maximizar a quantidade de trabalho não realizado; é essencial.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

11. As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

12. Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e então refina e ajusta seu comportamento de acordo.



OS 12 PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL

Na prática, os princípios básicos dos métodos ágeis são, por vezes, **difíceis** de se concretizar

DESAFIOS DOS PRINCÍPIOS ÁGEIS

Envolvimento do cliente depende da **disponibilidade** e representação efetiva de todos os stakeholders.



DESAFIOS DOS PRINCÍPIOS ÁGEIS

Algumas pessoas na equipe podem não se **adaptar** bem ao intenso envolvimento exigido pelos métodos ágeis.



DESAFIOS DOS PRINCÍPIOS ÁGEIS

Priorizar mudanças é desafiador em sistemas com **múltiplos stakeholders**, cada um com suas próprias prioridades.



DESAFIOS DOS PRINCÍPIOS ÁGEIS

Manter a **simplicidade** requer esforço adicional, especialmente sob pressão de prazos de entrega.



DESAFIOS DOS PRINCÍPIOS ÁGEIS

Grandes empresas podem enfrentar dificuldades ao mudar de uma **cultura** de processos informais para processos definidos.



2

MÉTODOS ÁGEIS

MÉTODOS ÁGEIS

- **Desenvolvimento Incremental:**
 - Pequenos incrementos e lançamentos frequentes para os clientes;
 - A cada duas ou três semanas.
- **Envolvimento dos clientes:** para feedback rápido sobre requisitos.
- **Minimização da documentação:** em favor de comunicação informal.



MÉTODOS ÁGEIS

Métodos ágeis são altamente eficazes no desenvolvimento de:

- Produtos, pequenos ou médios, feitos por empresas de software para venda;
- Sistemas personalizados dentro das empresas;
 - Com compromisso claro do cliente;
 - Pouca interferência externa de regras e regulamentos.

MÉTODOS ÁGEIS

São mais **adequados** ao desenvolvimento de aplicativos nos quais os **requisitos de sistema mudam rapidamente** durante o processo de desenvolvimento.

MÉTODOS ÁGEIS

Visam a **entregar rapidamente** o **software funcional** aos clientes, possibilitando que eles proponham alterações e novos requisitos para iterações posteriores do sistema.

CARACTERÍSTICAS DOS MÉTODOS ÁGEIS

- Ciclos de desenvolvimento curtos e iterativos (*Métodos Leves*) ;
- Equipes auto-organizadas;
- Designs mais simples;
- Refatoração de código;



CARACTERÍSTICAS DOS MÉTODOS ÁGEIS

- Desenvolvimento orientado por testes;
- Envolvimento frequente do cliente;
- Ênfase na criação de um produto de trabalho demonstrável a cada ciclo de desenvolvimento.



OBJETIVOS DOS MÉTODOS ÁGEIS

- Reduzir a burocracia do processo;
- Evitar trabalho de longo prazo de valor duvidoso;
- Minimizar a criação de documentação pouco utilizada.



Framework	Contribuidor Principal
Adaptive Software Development (ASD)	Jim Highsmith, Sam Bayer
Agile Modeling	Scott Ambler, Robert Cecil Martin
Agile Unified Process (AUP)	Scott Ambler
Disciplined Agile delivery	Scott Ambler
Dynamic systems development method (DSDM)	Jennifer Stapleton
Extreme Programming (XP)	Kent Beck, Robert Cecil Martin
Feature-driven development (FDD)	Jeff De Luca
Lean software development	Mary Poppendieck, Tom Poppendieck
Lean startup	Eric Ries
Kanban	Taiichi Ohno
Rapid application development (RAD)	James Martin
Scrum	Ken Schwaber, Jeff Sutherland
Scaled Agile framework - SAFe	Scaled Agile, Inc.

EXTREME PROGRAMMING (XP)

Histórias de Usuários
Valores
Critérios de Teste de Aceitação
Plano de Iteração

1. PLANEJAMENTO

2. PROJETO

Design Simples
Cartões CRC
Protótipos

3. CODIFICAÇÃO

Programação
em Pares

REFATORAÇÃO

Teste Unitário
Integração Contínua

4. TESTE

Teste de Aceitação

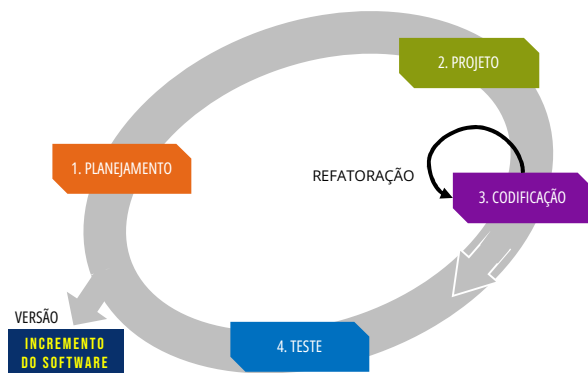
VERSÃO

**INCREMENTO
DO SOFTWARE**

Velocidade do
Projeto Computada

EXTREME PROGRAMMING (XP)

Framework ágil de desenvolvimento de software que visa produzir software de **maior qualidade** e **maior qualidade** de vida para a equipe de desenvolvimento.



EXTREME PROGRAMMING (XP)

- O nome foi cunhado por Beck (2000)
- A abordagem foi desenvolvida para impulsionar práticas reconhecidamente boas
- Exemplo: desenvolvimento iterativo, a níveis “extremos”.



EXTREME PROGRAMMING (XP)

O desenvolvimento incremental é sustentado por meio de **pequenos e frequentes releases** do sistema.



EXTREME PROGRAMMING (XP)

- Os **requisitos** são baseados em **cenários** ou **histórias de usuários**;
- Usadas como base para decidir a **funcionalidade** que deve ser incluída em um **incremento** do sistema.



EXTREME PROGRAMMING (XP)

O envolvimento do cliente é sustentado por meio do **engajamento contínuo do cliente** com a equipe de desenvolvimento.



EXTREME PROGRAMMING (XP)

- O **representante do cliente** participa do desenvolvimento;
- Responsável por definir os **testes de aceitação** para o sistema.



EXTREME PROGRAMMING (XP)

Pessoas não processos, são sustentadas por meio de:

- Programação em pares;
- Propriedade coletiva do código do sistema;
- Processo de desenvolvimento sustentável que não envolve horas de trabalho excessivamente longas.



EXTREME PROGRAMMING (XP)

As mudanças são aceitas por meio:

- De releases contínuos para os clientes;
- Do desenvolvimento *test-first* da Refatoração para evitar a degeneração do código;
- Integração contínua de nova funcionalidade.



EXTREME PROGRAMMING (XP)

A manutenção da simplicidade é feita por meio:

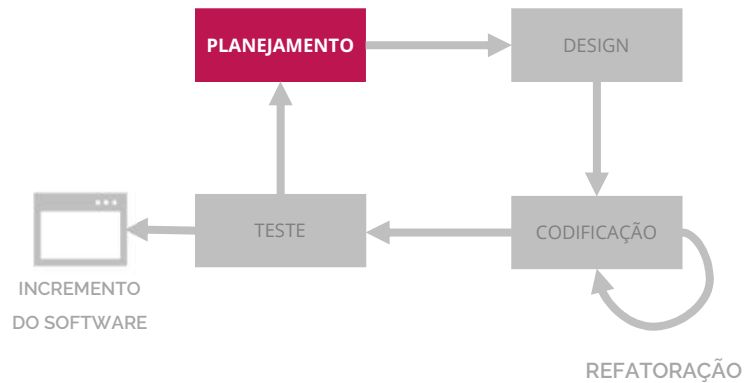
- Da Refatoração constante que melhora a qualidade do código,
- De projetos simples que não antecipam desnecessariamente futuras mudanças no sistema.



EXTREME PROGRAMMING (XP)

Escuta Ativa para:

- Entender o contexto de negócios do software;
- Ter uma visão ampla da saída necessária e dos principais recursos e funcionalidades.



EXTREME PROGRAMMING (XP)

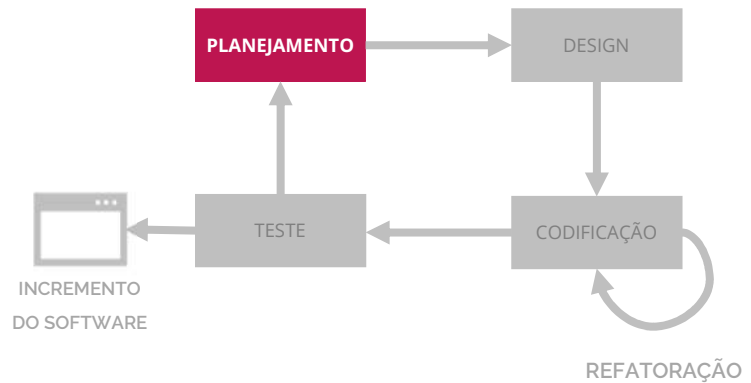
A **escuta** leva à criação de **histórias de usuários** que descrevem a **saída**, os **recursos** e a **funcionalidade necessários** para o software a ser construído.

Como um **viajante**,
eu quero **salvar todos**
os meus voos na minha
companhia aérea favorita
para que **eu possa**
acumular milhas aéreas

Como um **<papel>**,
eu quero **<meta/desejo>**
para que **<benefício>**

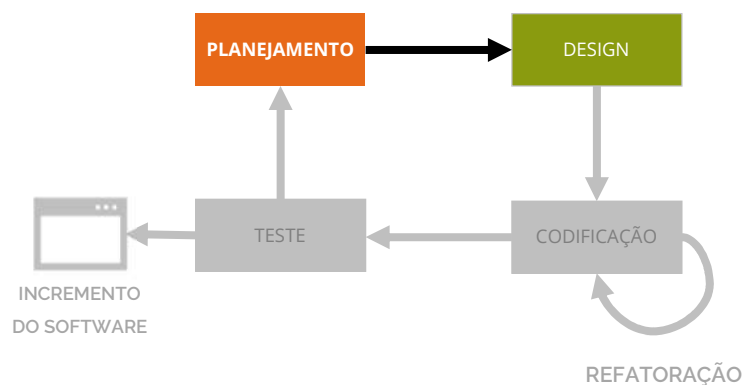
EXTREME PROGRAMMING (XP)

Clientes e desenvolvedores trabalham juntos para decidir como **agrupar histórias** no próximo lançamento (incremento de software) a ser desenvolvido pela equipe XP.



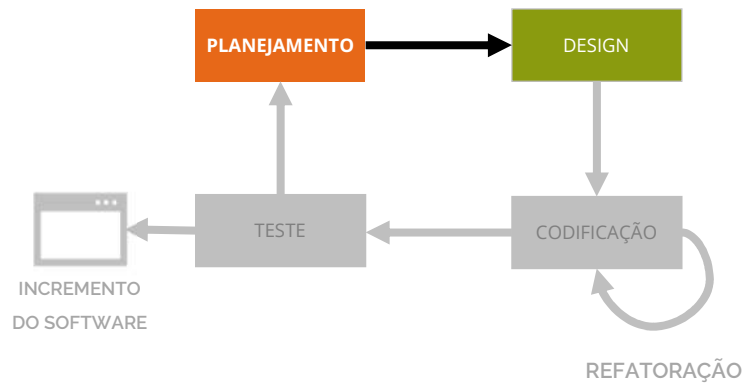
EXTREME PROGRAMMING (XP)

O design do XP segue rigorosamente o princípio KIS (*Keep it Simple* – Mantenha-o simples).



EXTREME PROGRAMMING (XP)

Um **design simples** é sempre preferido em vez de uma representação mais complexa.



EXTREME PROGRAMMING (XP)

O XP incentiva o uso de **cartões CRC** como um mecanismo eficaz para **pensar** sobre o software em um contexto **orientado a objetos**.

Class Sales	
Responsibility	Collaboration
• Knowledge • Behaviour • Operation • Promotion ...	• Partner • Clients ...

Class Transaction	
Responsibility	Collaboration
• Money Transfer • Auditing ...	• Card reader • Clients ...

Class Order	
Responsibility	Collaboration
• Price • Stock • Valid Payment ...	• Customers • Order line ...

Class Delivery	
Responsibility	Collaboration
• Item Identity • Check Receiver • Order No. • Total Qty ...	• Partner • Clients ...

Os cartões CRC são o único produto de trabalho de design produzido como parte do processo XP.

EXTREME PROGRAMMING (XP)

Os cartões CRC (class-responsability-collaborator) identificam e organizam as classes orientadas a objetos que são relevantes para o atual incremento de software.

Class Sales	
Responsibility	Collaboration
• Knowledge • Behaviour • Operation • Promotion ...	• Partner • Clients ...

Class Transaction	
Responsibility	Collaboration
• Money Transfer • Auditing ...	• Card reader • Clients ...

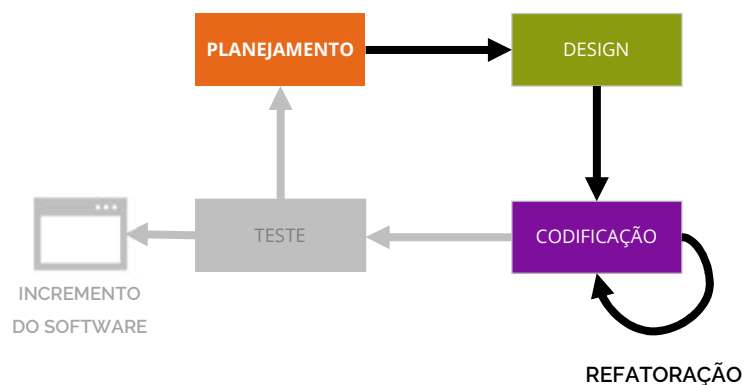
Class Order	
Responsibility	Collaboration
• Price • Stock • Valid Payment ...	• Customers • Order line ...

Class Delivery	
Responsibility	Collaboration
• Item identity • Check Receiver • Order No. • Total Qty ...	• Partner • Clients ...

Os cartões CRC são o único produto de trabalho de design produzido como parte do processo XP.

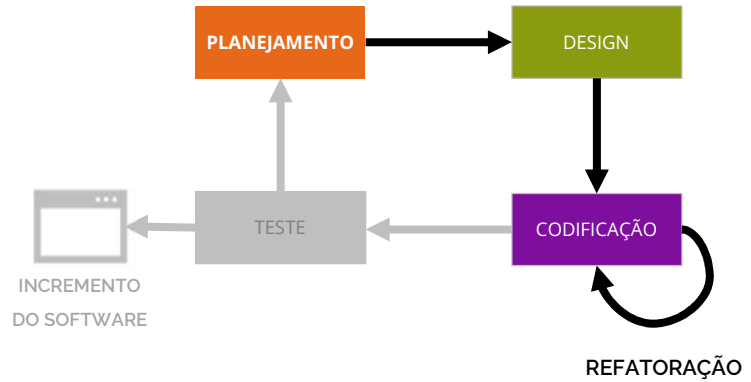
EXTREME PROGRAMMING (XP)

- A equipe não passa para o código imediatamente.
- A equipe desenvolve uma série de testes unitários para cada história a ser incluída na versão atual do software.



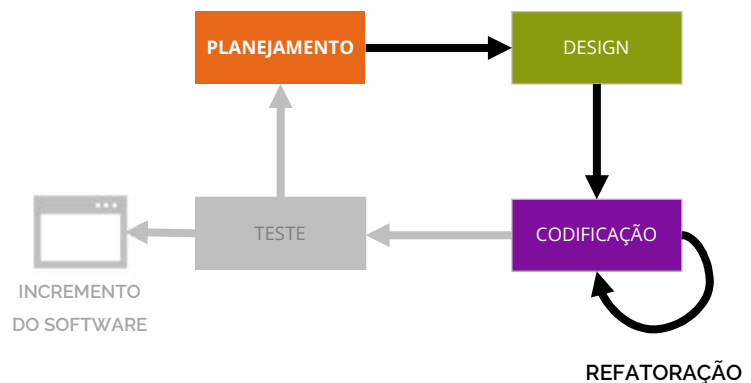
EXTREME PROGRAMMING (XP)

Os **testes unitários** ajudam a equipe a se **concentrar** no que precisa ser implementado para passar nos testes.



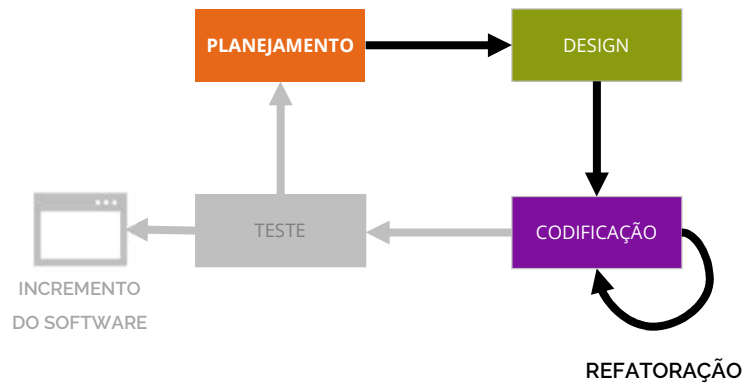
EXTREME PROGRAMMING (XP)

- Uma vez que o código esteja completo, é testado imediatamente;
- Proporcionando feedback instantâneo aos desenvolvedores.



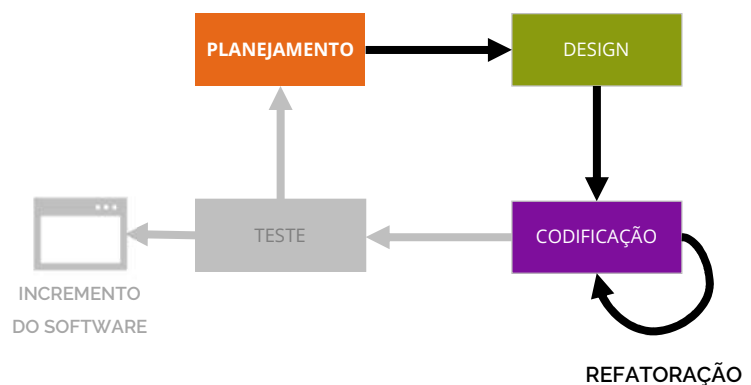
EXTREME PROGRAMMING (XP)

Um **conceito-chave** durante a atividade de codificação (e um dos aspectos mais comentados do XP) é a **programação em pares**.



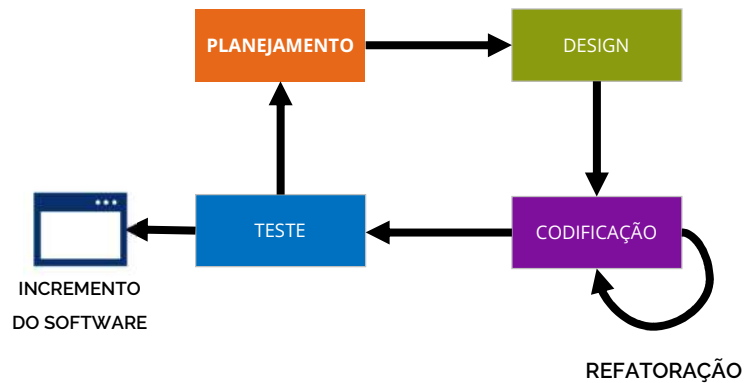
EXTREME PROGRAMMING (XP)

O XP recomenda que **duas pessoas** trabalhem juntas em uma estação de trabalho de computador para **criar código para uma história**.

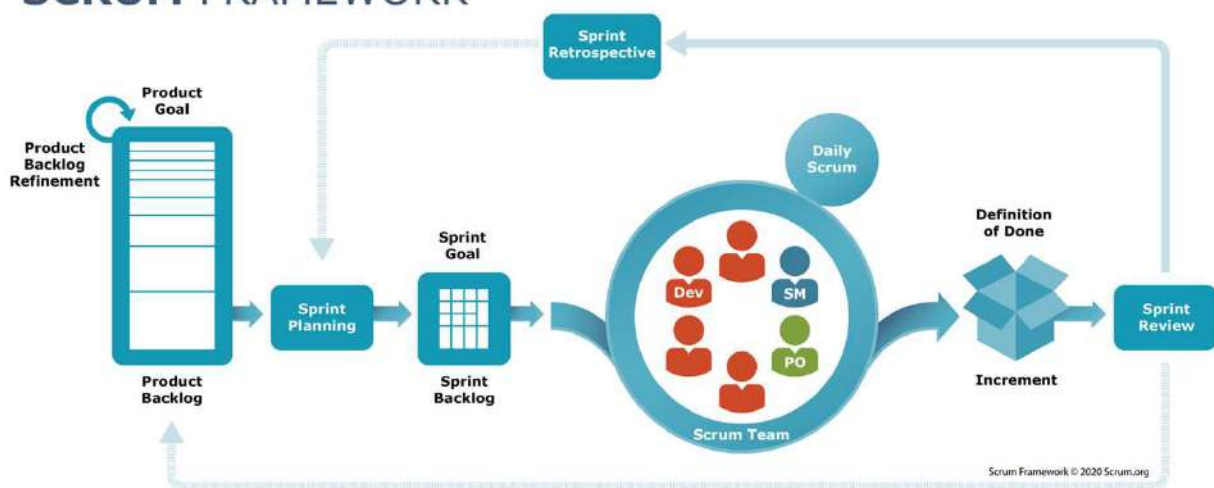


EXTREME PROGRAMMING (XP)

- Os testes de unidade criados devem ser implementados
- Usando uma estrutura que permita sua automatização
- Para que eles podem ser executados de forma fácil e repetida.



SCRUM FRAMEWORK



SCRUM

- Scrum é um método popular de desenvolvimento ágil de software.
- O nome é derivado de uma atividade em partidas de Rugby.



Scrum no Rugby é uma formação tática na qual os jogadores se juntam em uma disputa física pela posse da bola. Eles se alinham e se empurram, enquanto o jogador que coloca a bola no meio tenta empurrá-la para trás, enquanto os jogadores de cada equipe tentam fazer o contrário. O objetivo é recuperar a posse da bola ou mantê-la enquanto avançam pelo campo.

SCRUM FRAMEWORK

O framework Scrum foi concebido por Jeff Sutherland e sua equipe no início dos anos 1990.



SCRUM FRAMEWORK

Os princípios do Scrum
estão alinhados com o
manifesto ágil.

Estamos descobrindo maneiras melhores de desenvolver software,
fazendo-o nós mesmos e ajudando outros a fazerem o mesmo.

Através deste trabalho, passamos a valorizar:

Indivíduos e interações mais que *processos e ferramentas*
Software em funcionamento mais que *documentação abrangente*
Colaboração com o cliente mais que *negociação de contratos*
Responder a mudanças mais que *seguir um plano*

Ou seja, mesmo havendo valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

SCRUM FRAMEWORK

Os princípios do Scrum guia as
atividades de desenvolvimento
dentro de um processo que inclui
as seguintes atividades:



SCRUM FRAMEWORK

As tarefas de trabalho ocorrem em **períodos curtos e limitados** no tempo, chamados de **sprints**.



SCRUM FRAMEWORK

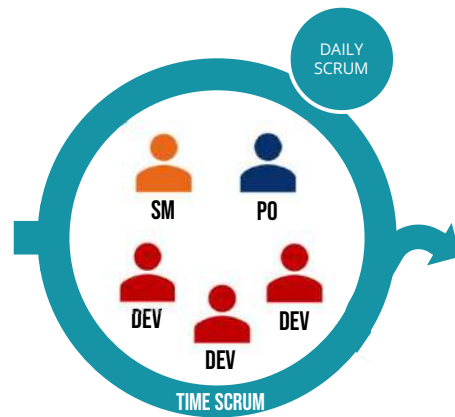
- O trabalho dentro de cada sprint é adaptado ao problema em questão;
- É definido e frequentemente modificado em **tempo real** pela equipe Scrum.



EQUIPE SCRUM

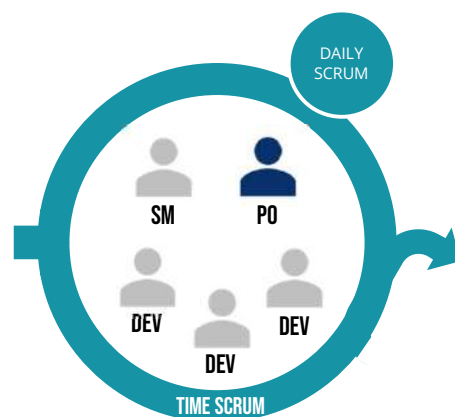
A equipe Scrum é uma equipe interdisciplinar auto organizável composta por:

- Um proprietário de produto (*Product Owner*)
- Um Scrum master
- Uma pequena equipe de desenvolvimento (3 a 6 pessoas).



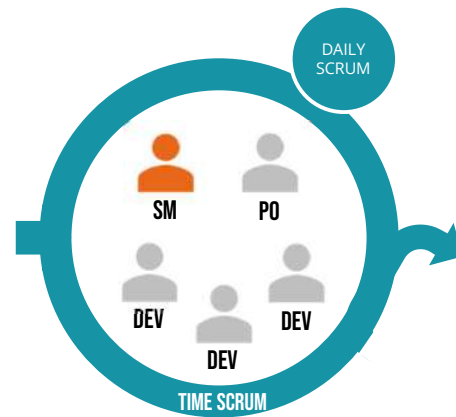
PRODUCT OWNER (PROPRIETÁRIO DO PRODUTO)

- Responsável por definir e priorizar os requisitos do produto;
- Representa os interesses do cliente e dos stakeholders;
- Define a visão do produto e toma decisões sobre o que deve ser desenvolvido.



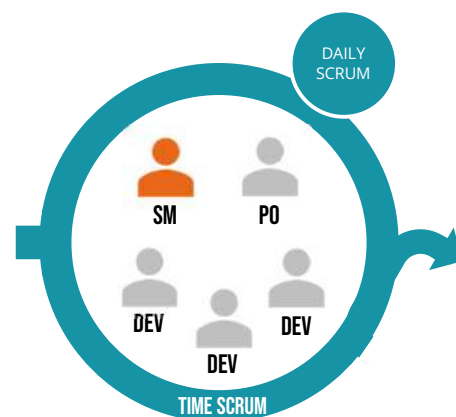
SCRUM MASTER (MESTRE SCRUM)

- Facilita o processo Scrum:
 - Removendo impedimentos
 - Garantindo que a equipe siga as práticas e os valores do Scrum.
- Ajuda a equipe a entender e implementar os princípios e práticas do Scrum.



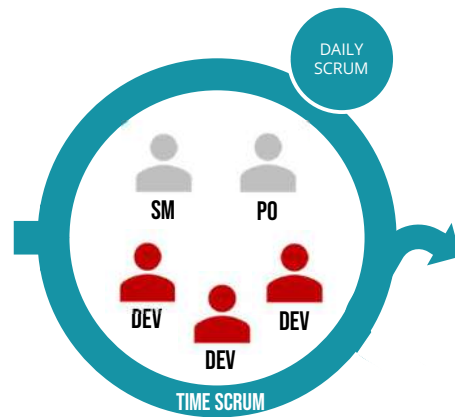
SCRUM MASTER (MESTRE SCRUM)

- Protege a equipe contra influências externas;
- Ajuda a resolver conflitos internos.



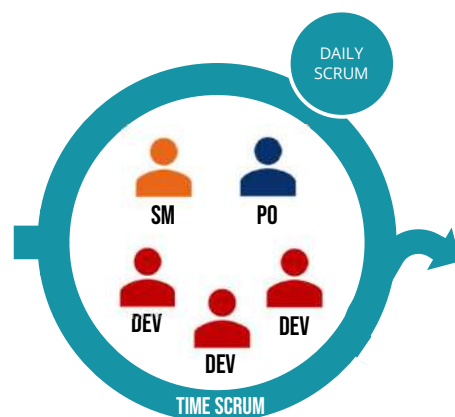
DEVELOPMENT TEAM (EQUIPE DE DESENVOLVIMENTO)

- É uma equipe multifuncional e auto-organizada que trabalha para entregar o produto.
- Responsável por transformar os itens do backlog da sprint em incrementos potencialmente entregáveis do produto.



DEVELOPMENT TEAM (EQUIPE DE DESENVOLVIMENTO)

Colabora diretamente com o *Product Owner* para entender os requisitos e com o *Scrum Master* para garantir um ambiente de trabalho eficaz.



DEVELOPMENT TEAM (EQUIPE DE DESENVOLVIMENTO)

Os principais artefatos do Scrum são:

- O backlog do produto
- O backlog da sprint
- O incremento do código.



BACKLOG DO
PRODUTO



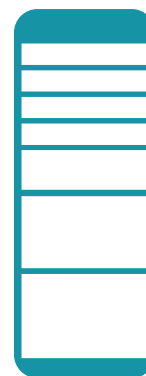
BACKLOG DA
SPRINT



INCREMENTO

BACKLOG DO PRODUTO

O backlog do produto é uma lista priorizada de todas as funcionalidades, requisitos, melhorias e correções que precisam ser feitas no produto.



BACKLOG DO
PRODUTO

BACKLOG DO PRODUTO

- É de responsabilidade do Product Owner gerenciar o backlog do produto
- Ele prioriza os itens com base no valor para o cliente e nas necessidades do negócio.



BACKLOG DO PRODUTO

- O backlog do produto é dinâmico e está sempre sujeito a alterações;
- Com novos itens sendo adicionados, removidos ou reordenados conforme necessário.



BACKLOG DA SPRINT

O backlog da sprint seleção do backlog do produto que a equipe de desenvolvimento concorda em trabalhar durante uma sprint específica.



**BACKLOG
DA SPRINT**

BACKLOG DA SPRINT

- É composto por uma lista de itens do backlog do produto que foram priorizados para aquela sprint;
- Com base na capacidade da equipe e nos objetivos definidos.



**BACKLOG
DA SPRINT**

BACKLOG DA SPRINT

- O backlog da sprint é congelado no início da sprint e só pode ser alterado em circunstâncias excepcionais;
- Exemplo: a descoberta de novas informações que afetam a sprint.



**BACKLOG
DA SPRINT**

INCREMENTO DO CÓDIGO

- O incremento do código é o resultado tangível do trabalho realizado durante uma sprint.
- Consiste em todas as funcionalidades desenvolvidas e testadas durante a sprint que atendem aos critérios de conclusão definidos.



INCREMENTO

INCREMENTO DO CÓDIGO

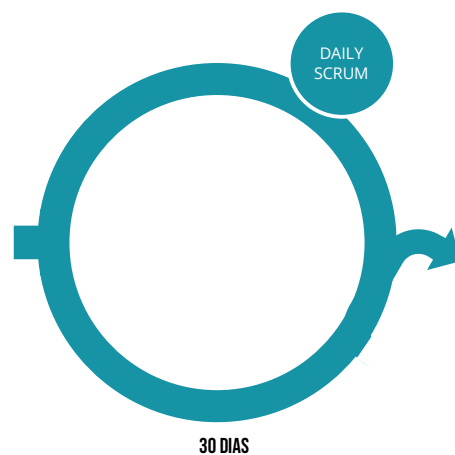
- Cada incremento do código é uma versão potencialmente entregável do produto;
- Que adiciona valor ao produto e pode ser lançada ou demonstrada ao cliente no final da sprint.



INCREMENTO

DEVELOPMENT TEAM (EQUIPE DE DESENVOLVIMENTO)

- O desenvolvimento prossegue ao dividir o projeto em uma série de períodos de desenvolvimento incremental, chamados de sprints;
- Cada sprint dura de 2 a 4 semanas.



REUNIÃO DE PLANEJAMENTO DA SPRINT

A reunião de planejamento (*Sprint Planning*), é um evento do framework Scrum que **marca o início de uma nova sprint no ciclo de desenvolvimento ágil de software.**

PLANEJAMENTO
DA SPRINT

REUNIÃO DE PLANEJAMENTO DA SPRINT

A reunião de planejamento (*Sprint Planning*), é um evento do framework Scrum que **marca o início de uma nova sprint no ciclo de desenvolvimento ágil de software.**

PLANEJAMENTO
DA SPRINT

REUNIÃO DE PLANEJAMENTO DA SPRINT

A Sprint Planning aborda os seguintes tópicos:

1. Por que esta Sprint é valiosa?
2. O que pode ser feito nesta Sprint?
3. Como o trabalho escolhido será realizado?

PLANEJAMENTO
DA SPRINT

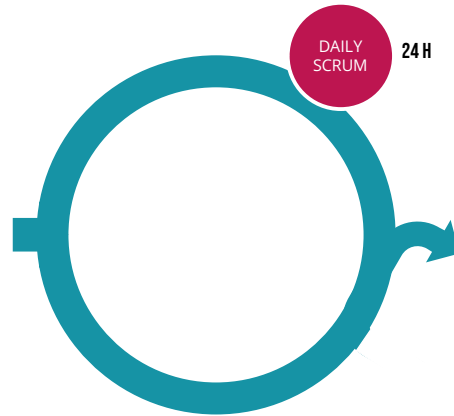
REUNIÃO DE PLANEJAMENTO DA SPRINT

- A reunião de planejamento de sprint é essencial para:
 - Alinhar as expectativas da equipe;
 - Definir metas claras;
 - Selecionar o trabalho a ser realizado durante a sprint.
- Garantir uma execução eficiente e focada nas necessidades do cliente.

PLANEJAMENTO
DA SPRINT

REUNIÃO DIÁRIA DO SCRUM

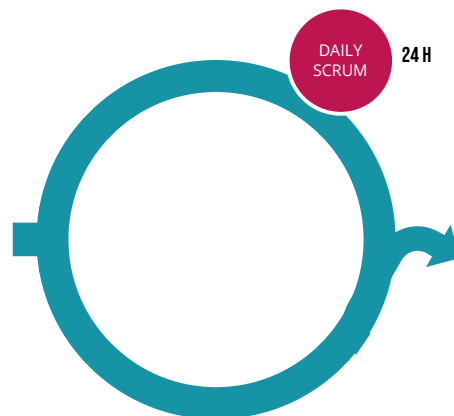
- A reunião diária do Scrum (*Daily Scrum Meeting*) é um evento de 15 minutos agendado no início de cada dia de trabalho.
- Permite que os membros da equipe:
 - Sincronizem suas atividades
 - Façam planos para as próximas 24 horas.



REUNIÃO DIÁRIA DO SCRUM

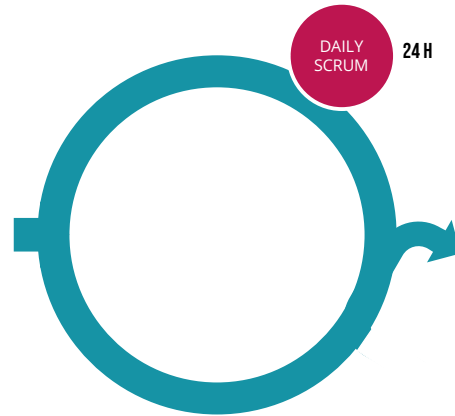
Três perguntas-chave são feitas e respondidas por todos os membros da equipe:

1. O que você fez desde a última reunião da equipe?
2. Quais obstáculos você está enfrentando?
3. O que você planeja realizar até a próxima reunião da equipe?



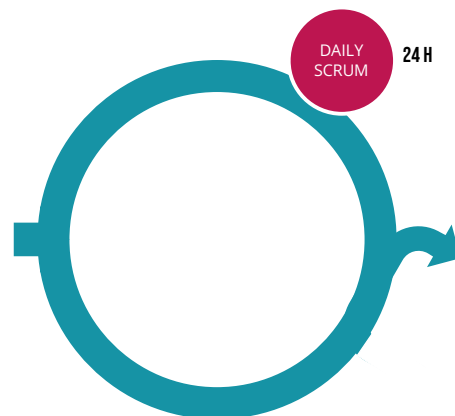
REUNIÃO DIÁRIA DO SCRUM

- O Scrum Master lidera a reunião e resolve obstáculos quando possível.
- Promove a identificação precoce de problemas e facilita a resolução fora do ambiente de reunião.



REUNIÃO DIÁRIA DO SCRUM

- Algumas equipes utilizam a reunião para declarar a conclusão de itens do backlog da sprint.
- Pode resultar no agendamento de uma demonstração do incremento concluído com o Product Owner quando todos os itens são considerados completos.



REUNIÃO DE REVISÃO DA SPRINT

- A Revisão da Sprint (*Sprint Review*) ocorre ao final da sprint, quando o time de desenvolvimento considera o incremento completo.
- Geralmente, a revisão da sprint tem uma duração máxima de 4 horas para uma sprint de 4 semanas.



REUNIÃO DE REVISÃO DA SPRINT

- Os participantes da reunião incluem o:
 - Scrum Master
 - Time de desenvolvimento
 - Product Owner
 - Stakeholders selecionados.
- Durante a revisão, o Product Owner pode aceitar ou não o incremento.
- O feedback fornecido pode influenciar o planejamento da próxima sprint.



REUNIÃO DE REVISÃO DA SPRINT

- A reunião de retrospectiva da sprint ocorre ao final de cada sprint no Scrum.
- Durante essa reunião, o time revisa o trabalho realizado na sprint anterior.
- São discutidos os pontos positivos e negativos da sprint, bem como oportunidades de melhoria.

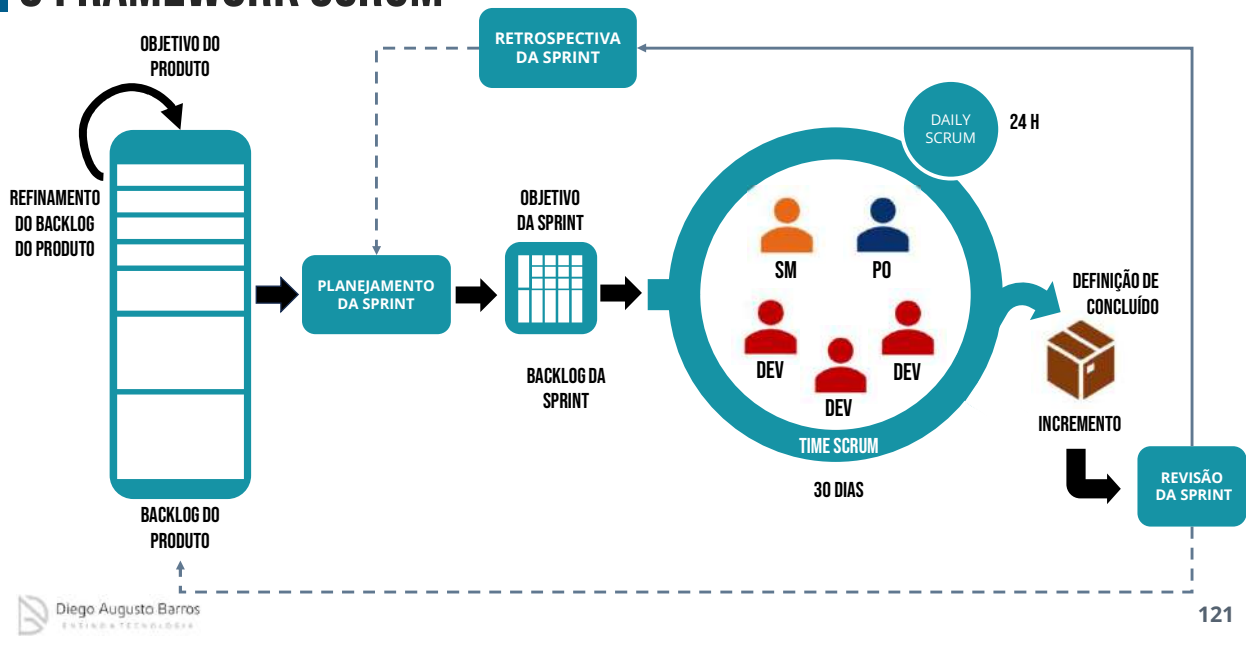
RETROSPECTIVA
DA SPRINT

REUNIÃO DE REVISÃO DA SPRINT

- O objetivo é aprender com as experiências da sprint anterior e fazer ajustes para aumentar a eficiência e a qualidade do trabalho na próxima sprint.
- O Scrum Master lidera a reunião, incentivando a participação de todos os membros do time.

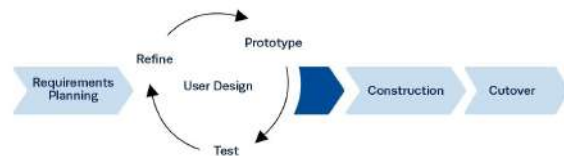
RETROSPECTIVA
DA SPRINT

O FRAMEWORK SCRUM



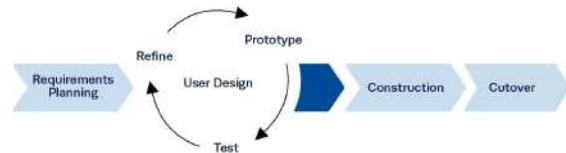
RAPID APPLICATION DEVELOPMENT (RAD)

- Aplicados principalmente em sistemas de negócios intensivos em dados.
- O método R.A.D. utiliza ferramentas especializadas de desenvolvimento de banco de dados.
- Essas ferramentas permitem que engenheiros de software desenvolvam, testem e implantem rapidamente novas ou modificadas aplicações de negócios.



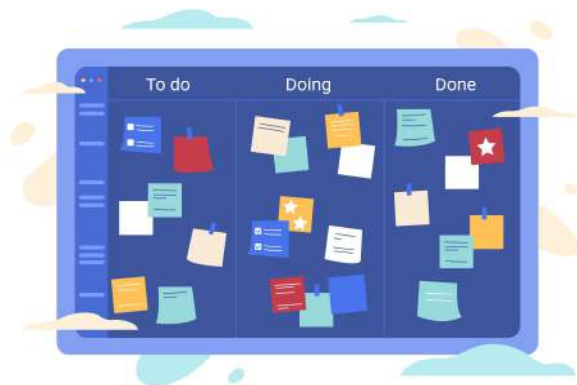
RAPID APPLICATION DEVELOPMENT (RAD)

- Aplicados principalmente em sistemas de negócios intensivos em dados.
- O método R.A.D. utiliza ferramentas especializadas de desenvolvimento de banco de dados.
- Essas ferramentas permitem que engenheiros de software desenvolvam, testem e implantem rapidamente novas ou modificadas aplicações de negócios.



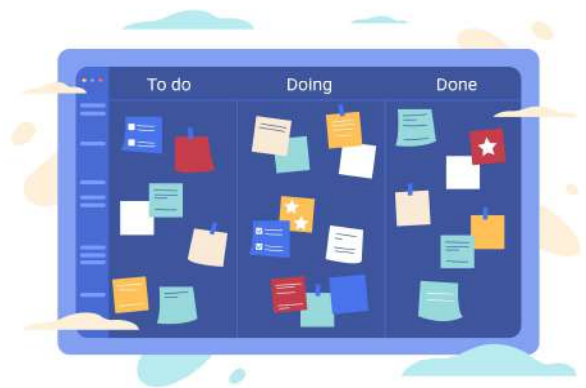
KANBAM

- Kanban é uma abordagem para gerenciar o trabalho de forma mais eficiente, originária do sistema de produção da Toyota.
- É um método visual para mapear, controlar e melhorar o fluxo de trabalho.



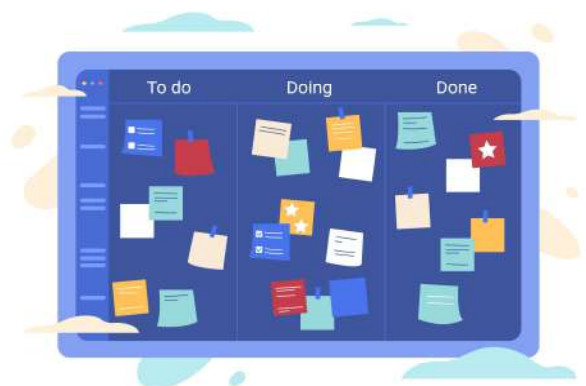
KANBAM

- Ele utiliza um quadro Kanban, que é dividido em colunas que representam diferentes estágios do processo de trabalho
- Os cartões que representam itens de trabalho individuais.



OBJETIVOS DO KANBAM

- Limitar a quantidade de trabalho em progresso (WIP) em cada etapa do processo
- Identificar gargalos e impedimentos
- Promover a colaboração e a melhoria contínua.



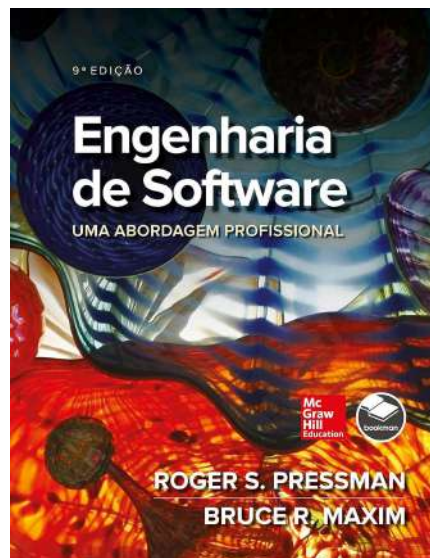
BIBLIOGRAFIA

SOMMERVILLE, Ian. **Engenharia de Software**. Cap. 2 – Processo de Software. pág. 18. Cap. 3 – Desenvolvimento Ágil de Software. pág. 38. Pearson, 2019.



BIBLIOGRAFIA

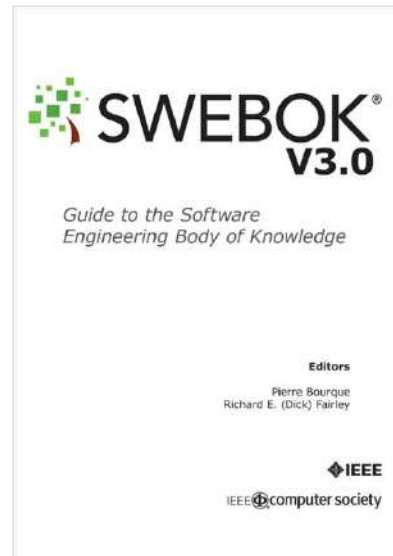
PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. Cap. 3 Agilidade e Processo, pág. 37. AMGH, 2021.



BIBLIOGRAFIA

BOURQUE, Pierre.; FAIRLEY, Richard E. (Dick).

SWEBOK V3.0. Chapter 9. Software Engineering Models and Methods, pág. 91. AMGH, 2021.



PROF. DIEGO AUGUSTO BARROS

 diegoaugustobarros.com

 [@diegoaugustobarros](https://www.linkedin.com/in/diegoaugustobarros)

 [@profdiegoaugusto](https://github.com/profdiegoaugusto)

Diego Augusto Barros é bacharel em Sistemas de Informação pela Pontifícia Universidade Católica de Minas Gerais (2012) e mestre em Ciência da Computação pela Universidade Federal de Minas Gerais (2015).

Sua pesquisa concentra-se nas áreas de Visualização de Dados e Interação Humano-computador, e investiga fatores cognitivos e perceptivos envolvidos na análise de grandes conjuntos de dados, que resultam em novos sistemas interativos para comunicação e análise visual.

Seus principais interesses nas áreas são: visualização de informação, Visual Analytics, métodos de avaliação de interfaces, interação com sistemas, tecnologias web, sistemas de informação, engenharia de software e informática na educação.